

Continuous Control for Vision-Based Lane Following: Benchmarking Deep Reinforcement Learning Algorithms

Anonymous Authors^{1*}

^{1*} Affiliations omitted for peer review.

Corresponding author(s). E-mail(s): [omitted@for.peer.review](#);

Abstract

Autonomous vehicles require robust decision-making and continuous control systems capable of handling the noise of vision-based perception. This paper presents a comprehensive study that addresses two primary research questions regarding Deep Reinforcement Learning (DRL) continuous lane-following control: benchmarking prominent model-free algorithms (PPO, DDPG, TD3, and SAC) under diverse training scenario variety, and systematically investigating how this training variety contributes to policy generalization and robustness. Additionally, we address a third question concerning the transition from ideal to realistic perception: evaluating the control impact of transitioning from ideal ground-truth observations to a Deep Learning-based perception pipeline. We maintain a unified observation space composed of coordinates in images and kinematic observations, decoupling perception features from the policy architecture. To obtain reliable coordinates in images from visual inputs, we utilize a custom fine-tuned YOLOP model, which provides highly accurate 2D coordinate extractions while maintaining real-time performance. We evaluate the control policies across 30 independent runs in CARLA simulator, demonstrating that while perception noise increases lateral deviation, the best-performing SAC agent successfully navigates complex curves at high speeds (reaching peaks of 90 km/h) without lane departures. This establishes a rigorous framework to assess both training techniques and perception-induced control degradation.

Keywords: Autonomous Driving, Deep Reinforcement Learning, Lane Following, Algorithm Benchmarking, YOLOP

1 Introduction

Autonomous Driving is advancing at an unprecedented pace, with increasingly complex tasks being tackled. Developing a fully autonomous vehicle requires breaking down the driving problem into a wide spectrum of specialized operational and tactical maneuvers. A robust decision-making module must be capable of dynamically responding to these diverse scenarios, making the design of such systems more critical than ever.

Over the past decade, numerous companies have heavily invested in autonomous vehicle technologies, including Tesla, Waymo, Cruise, NVIDIA, Mobileye (Intel), Baidu, Uber, and traditional automakers like Ford, GM, and Toyota. These efforts range from sensor innovation and simulation tools to full-stack self-driving platforms. Currently, most real-world deployments remain at SAE J3016 Levels 2–3 of autonomy, where human supervision is still required. Level 4 autonomy (full self-driving under constrained conditions) has seen limited deployment in cities like Phoenix and San Francisco, but widespread adoption remains constrained by challenges in perception, planning, safety guarantees, and legal frameworks.

Establishing truly autonomous vehicles is expected to profoundly reshape transportation. Instead of individual vehicle ownership, we may see fleets of shared autonomous taxis offering safer, more efficient, and more accessible mobility. This shift has the potential to reduce traffic congestion, improve road safety, lower emissions, and expand mobility to people with disabilities or the elderly. However, for this transformation to fully take hold, it is essential to solve key technological hurdles, including decision-making under uncertainty. Addressing these challenges is vital for executing typical tactical maneuvers and operational driving tasks, such as autoparking, lane following, cruise control, highway merging, and overtaking.

In the SOTA, lane-following strategies are generally structured around two primary design paradigms: *end-to-end* (E2E) architectures and *modular* pipelines. E2E systems directly map high-dimensional raw sensory inputs to physical actuator commands using deep neural networks. Conversely, modular systems decompose the complex driving task into specialized subsystems. This modular architecture is favored by prominent open-source autonomous driving suites, such as those developed by the Autoware Foundation¹, as it enables independent safety validation, subsystem replacement, and specialized tuning. In modular pipelines, the system is typically simplified into three distinct sequential stages:

- **Perception:** This stage is responsible for capturing sensory observations (such as raw images or LiDAR point clouds) and reconstructing relevant environmental features, including drivable area boundaries and lane-marking structures.
- **Path Planning:** Utilizing the perception outputs, this phase computes a reference trajectory, target speed profiles, or local waypoint coordinates.
- **Control:** Finally, this subsystem determines the continuous control actions (such as steering, throttling, and braking) necessary to accurately execute the planned trajectory while adhering to the vehicle’s physical kinematic constraints.

¹<https://www.autoware.org/>

In this work, we focus strictly on the *control* module of a decoupled, modular pipeline, specifically targeting the continuous steering and acceleration commands required for the fundamental lane-following task. This decoupled design enables us to split our research into two key sequential phases:

- **Isolated Control Benchmarking (Phase 1):** Comparing the continuous control capabilities, sample efficiency, and stability of PPO, DDPG, TD3, and SAC under ideal, noise-free ground-truth observations.
- **Perception-Control Evaluation (Phase 2):** Evaluating the resulting control impact when deploying these trained policies on waypoints derived from a fine-tuned YOLOP vision backbone.

Specifically, we completely isolate the steering and acceleration policy from raw image pixels by defining a unified, high-level observation space. This observation space is composed of localized coordinates in images representing the centerlane, combined with the vehicle’s own kinematic observations (specifically, absolute velocity, speed-matching error, and current steering angle). Since our work is restricted to continuous single-lane tracking, we simplify the Path Planning phase by setting the target path to always be the lane centerlane. Under the Phase 1 Ground-Truth setup, these centerlane waypoints are queried directly from CARLA, while under the Phase 2 Deep Learning perception setup, they are calculated by processing visual lane boundaries detected from camera images. Since the target path is always the lane center, we project these future waypoints relative to the vehicle’s local frame and feed them directly to the policy network, bypassing the need for a complex planning module.

While classical methods such as Model Predictive Control remain relevant for vehicle control, machine learning techniques including Imitation Learning and Deep Learning are gaining significant interest for training adaptive control policies in autonomous driving. Specifically, Deep Reinforcement Learning (DRL) extends standard Reinforcement Learning by using deep neural networks to approximate complex policies and value functions, enabling its application in high-dimensional, nonlinear driving tasks. Compared to classical controllers (e.g., PID, Pure Pursuit), DRL adapts autonomously without explicit modeling assumptions; compared to Imitation Learning, DRL can exceed expert performance and handle rare, safety-critical scenarios; and compared to Model Predictive Control, DRL operates without dependence on accurate vehicle models. These advantages make DRL uniquely well-suited for continuous, context-dependent control, such as the lane-following task addressed in this work.

However, the rapid adoption of DRL has outpaced its systematic evaluation. The literature is replete with applications of DRL algorithms, yet the rationale for selecting one over another is often ad-hoc or weakly justified. Furthermore, the impact of critical training methodologies and the diversity of training scenarios on final policy performance and generalization is seldom rigorously benchmarked, creating a critical knowledge gap for researchers and practitioners.

To address these gaps, this work is structured around three core research questions, which also represent the primary scientific contributions of this paper:

- **Continuous DRL Benchmarking (RQ1):** Benchmarking prominent model-free algorithms (PPO, DDPG, TD3, and SAC) to determine the optimal trade-off

between safety, tracking precision, and sample efficiency for continuous lane-centering control.

- **Training-Variety Analysis (RQ2):** Systematically investigating how training scenario variety (such as track layouts and speed randomizations) contributes to policy generalization and robustness on unseen circuits.
- **Perception-Control Integration (RQ3):** Evaluating how transitioning from ideal ground-truth observations to a fine-tuned visual YOLOP backbone affects control stability, quantifying safety-speed trade-offs under realistic noise and latency constraints.

To answer these questions, we establish a high-fidelity simulation and evaluation framework. The validation of modern autonomous vehicle algorithms increasingly relies on advanced simulation environments; among platforms like AirSim, GAZEBO, and LGSVL, this study employs the CARLA simulator [1], which has also emerged as the primary platform for prestigious industry benchmarks like the CARLA Autonomous Driving Leaderboard. CARLA’s realistic vehicle dynamics, support for stochastic environmental agents, and diverse weather conditions, coupled with full access to simulator ground truth, provide the most robust environment for developing and evaluating complex DRL-based control policies.

Within this environment, we implement our training and evaluation pipelines using the standardized RL-Studio and BehaviorMetrics frameworks [2]. To measure control quality, we compile a comprehensive set of performance and safety metrics, including success rate (percentage of completed routes), rate of collisions with environmental objects, lane boundary invasions, average longitudinal speed, and mean lateral deviation from the lane centerlane. In this work, we employ an experimental setup that does not include dynamic obstacles. While we acknowledge that the absence of other vehicles represents a limitation in terms of scenario complexity, this simplification is intentionally chosen to isolate and rigorously benchmark the agent’s core control and continuous decision-making capabilities, avoiding confounding variables from multi-agent interactions.

Furthermore, the benchmarking analysis of our four candidate model-free DRL algorithms (PPO, SAC, DDPG, and TD3) is supported by a comprehensive ablation study. This ablation study systematically investigates the role of training scenario variety to determine how training-set diversity influences final control robustness, safety, and policy generalization on unseen test circuits.

The remainder of this article is organized as follows. Section 2 reviews related work in deep reinforcement learning for autonomous driving. Section 3 describes the experimental setup, problem formulation, and the DRL algorithms employed. Section 4 presents the benchmarking results, including a detailed behavioral analysis and the ablation study on training diversity. Finally, Section 5 summarizes the main findings and outlines directions for future research.

2 Related Work

2.1 Evolution of Decision-Making and Control Paradigms

Autonomous driving decision-making has historically relied on a broad range of paradigms. Among early approaches, Bouchard et al. [3] proposed a rule-based behavior planner grounded in formal logic for structured environments, while Bae et al. [4] implemented a finite state machine-based behavioral planner to explicitly manage urban driving states such as cruising and stopping. These offer high interpretability but often scale poorly as environmental complexity increases due to the impossibility of anticipating every driving scenario. To overcome these limitations, machine learning approaches, particularly imitation learning, have been widely adopted to learn end-to-end control from expert demonstrations. For instance, Codevilla et al. [5] introduced conditional imitation learning to enable models to follow high-level navigational commands; Cheng et al. [6] presented PLUTO, an imitation learning-based planning framework designed to improve high-fidelity trajectory prediction and decision-making performance; and Moncalvillo and et al. [7] adapted imitation learning techniques to account for the physical constraints imposed by Ackermann steering geometry. While these models achieve strong performance under nominal conditions, they frequently struggle with generalization when encountering out-of-distribution events or safety-critical edge cases not present in the training dataset.

To address the challenges of generalization and long-term planning under uncertainty, reinforcement learning, and specifically deep reinforcement learning, has emerged as a central paradigm. By leveraging deep neural networks to approximate policies and value functions, deep reinforcement learning enables effective learning from rich sensory inputs and explicit optimization of long-term behavior. Recent surveys document this evolution: Zhao et al. [8] categorized deep reinforcement learning applications in autonomous driving and discussed persistent safety challenges, while Wu et al. [9] focused specifically on behavior planning in complex traffic interactions. Concurrently, several benchmark environments have been proposed to standardize agent evaluation; Lavington et al. [10] introduced TorchDriveEnv to facilitate training against reactive non-playable characters, and Liu et al. [11] developed a benchmark specifically targeting intersection scenarios. Comprehensive reviews further contextualize these developments: Kiran et al. [12] examined state-space representation and reward shaping strategies, Prasuna and Potturu [13] discussed sample efficiency and the sim-to-real gap, and Singh et al. [14] explored approaches for safer deployment in real-time environments. As a result, deep reinforcement learning now underpins many end-to-end and modular autonomous driving systems.

2.2 Task-Specific Control and Environmental Adaptability

Beyond foundational control, deep reinforcement learning has been applied to a wide array of increasingly complex autonomous driving tasks. For intersection navigation, Elallid et al. [15] demonstrated a DRL-based approach aimed at reducing delays in unsignalized intersections, Shankar et al. [16] evaluated reinforcement learning agents in uncontrolled intersection scenarios representative of Indian road conditions, and

Spatharis and Blekas [17] applied multi-agent reinforcement learning to improve coordination and traffic efficiency through decentralized decision-making. Furthermore, Jayawardana et al. [18] emphasized eco-driving strategies in multi-agent contexts using the INTERSECTIONZOO benchmark, and Cederle et al. [19] proposed a distributed coordination approach to reduce reliance on centralized intersection infrastructure. In the context of lane changing, Wang et al. [20] established a benchmark evaluating how different algorithms balance safety and traffic flow efficiency during merging maneuvers. Similarly, for car-following and freeway decision-making, Liu et al. [21] focused on high-speed stability, Chen et al. [22] introduced the FollowNet dataset and benchmark for modeling and evaluating car-following behavior and safe headway control, and Marin et al. [23] developed a deep reinforcement learning framework for safe and efficient autonomous car-following, balancing collision avoidance and traffic flow performance. Robustness to environmental factors has also been explored: J et al. [24] empirically established baselines for LiDAR degradation in rain and fog, Almaliloglu et al. [25] presented a positioning system fusing sensory inputs across varying weather conditions, Lee et al. [26] proposed a hybrid deep learning co-registration system, and Elallid et al. [27] integrated meteorological data to refine decision-making in variable weather. While these studies confirm the versatility of RL-based approaches, their diversity in tasks, environments, and evaluation protocols further reinforces the need for standardized, task-focused benchmarks that isolate core decision-making and control capabilities.

2.3 Decoupled Control Architectures and Vision-Based Perception

In complex continuous-control problems, reinforcement learning is frequently integrated into hierarchical or decoupled architectures. For example, Qiao et al. [28] proposed a hierarchical architecture in which a high-level planner selects maneuvers and a low-level controller executes them, improving modularity and interpretability. From a perception standpoint, vision-based setups remain common and cost-effective. Indeed, unlike high-end multi-sensor configurations that integrate LiDAR, radar, and multiple cameras—which are highly accurate but prohibitively expensive and computationally demanding—a single front-facing camera setup remains highly attractive for practical vehicle deployment due to its low hardware cost, simpler integration, and lower real-time processing overhead. Geiger et al. [29] established the KITTI benchmark, which laid foundational groundwork for stereo vision and 3D object detection in autonomous driving research. More recently, Busch et al. [30] enhanced single-camera Bird’s-Eye-View perception by leveraging multi-camera data during training, and Unger et al. [31] developed a framework providing a holistic 360-degree environmental representation to support decision-making. Lightweight detection architectures also play a crucial role; Wu et al. [32] introduced the YOLOP network for simultaneous traffic object detection and drivable area segmentation, Howard et al. [33] tailored MobileNet convolutional architectures for embedded vision, and Liu et al. [34] offered fast 2D object detection suitable for real-time driving via the Single Shot MultiBox Detector.

2.4 Continuous Lane-Following Control Benchmarks

A significant portion of the foundational literature focuses on the specific task of lane following, which serves as a critical testbed for continuous control. Studies in this area have targeted different aspects of control optimization: Hua et al. [35] refined the exploration strategy of DDPG to improve lateral control stability, Liu et al. [36] proposed an entropy-regularized controller maintaining precision at high speeds, and Sabbir [37] compared PPO and Double Deep Q-Networks, reporting stronger performance of PPO in continuous control settings. Despite widespread adoption, systematic comparisons among deep reinforcement learning algorithms remain relatively limited in autonomous driving contexts and often yield varying results. Representative approaches include PPO, originally introduced by Schulman et al. [38] using a clipped surrogate objective; DDPG, adapted for continuous action spaces by Lillicrap et al. [39]; TD3, proposed by Fujimoto et al. [40], which was designed to mitigate overestimation bias in actor-critic methods; and SAC, introduced by Haarnoja et al. [41], which is an entropy-regularized algorithm designed to balance exploration and exploitation. While some studies evaluate these methods under restricted conditions, such as Evans et al. [42], who compared deep reinforcement learning architectures in the context of autonomous racing with lap time as a primary metric, or Li et al. [43], who analyzed failure modes on consecutive sharp turns—broader analyses increasingly suggest that stability-enhanced actor-critic methods exhibit improved robustness. Specifically, Liu [44] conducted a systematic evaluation of actor-critic algorithms across standard continuous control benchmarks, Xue and et al. [45] compared reinforcement learning architectures specifically on lane-changing control stability, and Park et al. [46] evaluated algorithms under urban driving challenges and regulatory constraints within the CARLA simulator. This study directly builds upon these comparative foundations, utilizing a standardized simulation setup to analyze the performance and stability of prominent deep reinforcement learning algorithms under both perfect observations and realistic visual constraints.

3 Materials and Methods

3.1 Frameworks

The experimental setup leverages a robust training and evaluation pipeline centered on the open-source RL-Studio framework [47]² running on Ubuntu Linux 20.04 LTS. During the training phase, RL-Studio utilizes standardized, state-of-the-art DRL algorithm implementations from the Stable-Baselines3 library (version 2.4.1) [48]³, which is developed using PyTorch (version 2.2.0) with CUDA 12.4 support. This pipeline connects directly to the CARLA simulator (version 0.9.15) [1]⁴ via its native Python API in synchronous mode. This guarantees a fixed simulation frequency of 10 frames per second (with a fixed time-step $\Delta t = 0.1$ s), ensuring fast, deterministic data collection over millions of steps without communication bottlenecks.

²<https://github.com/JdeRobot/RL-Studio>

³<https://github.com/DLR-RM/stable-baselines3>

⁴<https://carla.org/>

Conversely, for policy evaluation and performance analysis, BehaviorMetrics is employed [2]⁵. BehaviorMetrics is a specialized GUI-based tool designed to assess and monitor autonomous driving control policies. For our lane-following control evaluation, it primarily tracks and computes key driving variables such as success rate, mean lateral deviation (cross-track error), lane boundary invasions, collisions, and average speed, which are explained in detail inside the evaluation protocol in Section 3.4.

During these evaluation runs, the architecture shifts to a ROS-based (Robot Operating System Noetic) [49]⁶ communication bridge. The control policy is executed within an active ROS node, where camera frames, localized waypoints, and kinematic states are published and subscribed to dynamically. This ROS bridge injects realistic, asynchronous message-passing latencies and scheduling jitter into the control loop. Incorporating these communication characteristics helps bridge the sim-to-real gap, validating that the trained control policies remain robust when deployed on actual distributed hardware topologies.

3.2 Problem Formulation and Setup

To thoroughly analyze the continuous control system, we distinguish between two operational configurations: the **Training Setup** and the **Inference Setup**.

The **Training Setup** (illustrated in Figure 1) is designed for policy optimization under safe, centralized, and synchronous simulation. During this phase, the system operates in a closed loop with the CARLA simulator. The DRL agent interacts with the environment by executing continuous actions, and receiving updated observations and reward feedback. During training, we utilize the "Perfect Teacher" technique: both the centerlane deviation in meters and the lane invasion signal, used to calculate step-by-step rewards, are extracted directly from CARLA's noise-free simulator ground truth. This ensures clean, uncorrupted learning gradients, allowing the policy network's weights to converge efficiently across different DRL algorithms (PPO, DDPG, TD3, and SAC).

Conversely, the **Inference Setup** (illustrated in Figure 2) represents the modular, decoupled control architecture deployed for real-time testing. During inference, the learned policy weights are frozen within the *AgentPolicyNetwork*, and the system operates asynchronously over the ROS communication bridge to simulate actual hardware constraints. Rather than relying on a single, fixed configuration, the system utilizes a real-time **Perception Stage** which, depending on the active evaluation phase, either queries noise-free coordinates (Phase 1, Ground-Truth) or processes raw camera frames through our custom fine-tuned YOLOP vision network (Phase 2, Deep Learning perception) to dynamically compute the local 2D coordinates of the lane centerlane waypoints. These coordinates are then combined with the vehicle's current kinematic observations to form the observation vector $\mathbf{s}_t$, which is fed into the *AgentPolicyNetwork*. Finally, the resulting continuous control commands are published directly to the vehicle's steering and throttle/brake actuators, closing the perception-control loop without calculating rewards.

⁵<https://github.com/JdeRobot/BehaviorMetrics>

⁶<https://www.ros.org/>

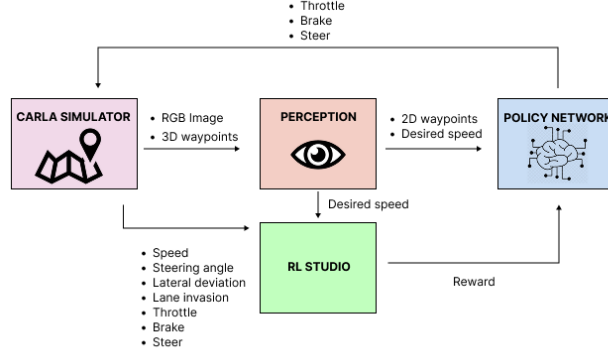


Fig. 1 System architecture during the training setup, showing ground-truth supervision and closed-loop reward updates.

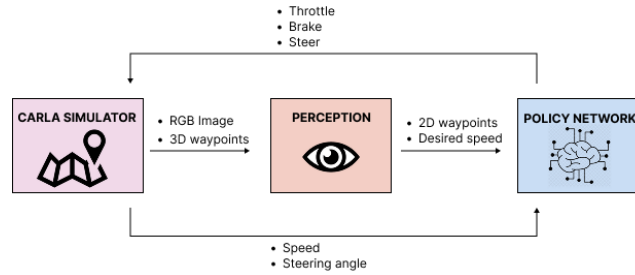


Fig. 2 System architecture during the inference setup, showing the decoupled perception backbone and frozen AgentPolicyNetwork.

3.2.1 Action Space

The vehicle’s actions are continuous controls applied to the vehicle dynamics model:

- **Throttle/Brake Action:** A continuous value ranging from -1 (full brake) to 1 (full throttle).
- **Steering Action:** A continuous value ranging from -0.5 (left) to 0.5 (right). While the maximum physical steering range in CARLA is $[-1, 1]$, we constrained the agent’s output range for two primary reasons: first, to enforce smoother, less aggressive steering behavior typical of high-speed maneuvers; and second, to accelerate initial policy acquisition by forcing the agent to focus on fine-grained lateral control. This choice is supported by initial hyperparameter sweeps which confirmed that the narrower range led to faster and more stable convergence.

3.2.2 Observation Space and Feature Extraction

To establish a fair and rigorous comparison, we employ an identical, unified observation space structure for both the ideal ground-truth benchmarking phase and the vision-based perception phase. This observation vector, $\mathbf{s}_t$, decouples perception features

from the neural network policy by feeding high-level geometric and kinematic vectors into the model:

- **Future Waypoints ($\mathbf{w}_{1..10}$):** A sequence of the next ten waypoints representing the estimated centerlane. Specifically, these are represented as coordinates in images, specifically 2D pixel coordinates (u_i, v_i) on the vehicle’s camera sensor plane. To provide temporal context and implicit acceleration data, these coordinates are buffered across the last 3 consecutive frames. This unified image-plane observation space ensures that the policy network’s input structure remains identical, regardless of the underlying perception source. Depending on the active experimental setup, these coordinates are calculated through two distinct extraction techniques:
 1. *Ground-Truth Perception:* The 10 future waypoints are queried as 3D geometric coordinates (x_i, y_i, z_i) in the simulator’s global frame relative to the vehicle, and are mathematically projected onto the 2D camera image plane using the camera’s intrinsic and extrinsic calibration matrices to obtain their 2D pixel coordinate representation (u_i, v_i) (as illustrated in Figure 3), establishing an ideal, noise-free baseline.
 2. *DeepLearning-based Perception:* The 10 waypoints are dynamically computed from the onboard vehicle camera. Raw camera images are fed into our custom fine-tuned YOLOP model to segment the drivable area and lane markings. These raw segmentation masks are then post-processed via customized mathematical morphology and skeletonization to directly calculate the 2D image-plane coordinates (u_i, v_i) of the centerlane (as illustrated in Figure 5).

Because both techniques utilize the exact same observation space structure, we can directly evaluate the performance degradation caused solely by perception noise, without modifying the underlying DRL network configurations or reward function.

- **Kinematic Observation:** This includes the steering angle (θ_{steer}), current throttle/brake commands, and a *Hybrid Velocity State Representation*:

$$\mathbf{s}_{\text{velocity}} = [v, (v_{\text{goal}} - v)]$$

where v is the vehicle’s absolute linear velocity and $v_{\text{goal}} - v$ is the direct velocity control error. Providing both the absolute speed (critical for capturing non-linear aerodynamic drag and tyre-friction physics) and the explicit error (allowing early layers to bypass learning subtraction, acting as a "Neural PID") speeds up training convergence by approximately three times. To ensure smooth and safe control, the target speed (v_{goal}) undergoes a custom *Two-Stage Filtering Pipeline* within the environment wrapper before being injected into the observation space and the reward function, preventing sudden, jerky throttle/brake fluctuations caused by frame-by-frame waypoint noise: (1) a *Sliding Window Moving Average* queue capped at 3 frames to smooth out high-frequency spikes while guaranteeing zero mechanical lag, and (2) an *Asymmetric Exponential Moving Average (EMA)* filter designed to mimic human driving safety instincts. If the target velocity decreases (e.g., approaching a sharp curve), the filter uses a high coefficient ($\alpha_{\text{decel}} = 0.85$) to instantly apply

a heavy weight on the new lower target, commanding rapid braking for safety. Conversely, if the target velocity increases, it uses a low coefficient ($\alpha_{\text{accel}} = 0.15$) to smooth the acceleration ramp out over consecutive steps, preventing wheel slip and ensuring passenger comfort.

- **Action History ($\mathbf{a}_{t-1}$):** The values of the immediately preceding throttle/brake and steer actions. Including these past actions encourages the agent to learn smooth trajectories and avoids sudden, erratic changes in control inputs [50].

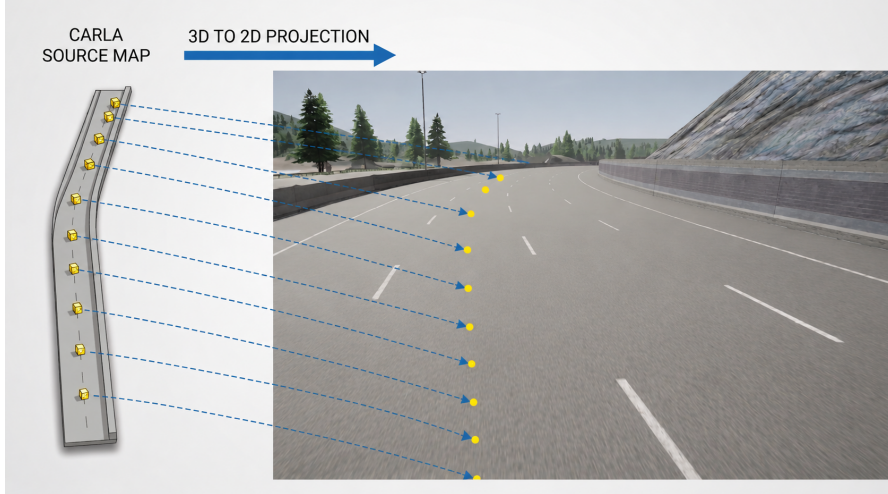


Fig. 3 Visualization of the centerlane waypoints projected directly on the 2D camera image plane using the simulator ground-truth coordinates.

3.2.3 Reward Function

The agent’s reward function, R_t , is engineered to prioritize safety (lane centering), efficiency (optimal speed), and stability (smooth control) simultaneously. Although the reward components operate in different physical units (e.g., speed and steering rates), all terms were empirically scaled during preliminary tuning to ensure comparable magnitudes and prevent the dominance of any single component.

We employ a “Perfect Teacher” reward strategy, which decouples the agent’s observation space from its feedback mechanism. While the DRL agent operates under realistic, noisy visual constraints (receiving only YOLOP-derived local coordinates), its reward function remains grounded in the CARLA simulator’s perfect ground truth. This strategy offers several structural advantages:

- **Noise-Free Supervision:** By using perfect ground truth for rewards, we ensure that the learning signal is consistently accurate, preventing the agent from being penalized for perception errors that are outside its control.
- **Performance Alignment:** The agent is guided toward the true optimal path (the lane center), even if its current perception is noisy. This allows us to isolate how

effectively the agent can learn to compensate for perception noise to reach the ideal state.

- **Evaluation Integrity:** Since the goal remains identical to the ground-truth benchmark, we can objectively measure the performance drop caused solely by the switch from GT observations to YOLOP observations.

The reward function, R_t , is mathematically formulated as:

$$R_t = \begin{cases} -20 & \text{if done} \\ R_{\text{positive}} - R_{\text{steer_rate}} - P_{\text{over}} - P_{\text{stagnation}} & \text{otherwise} \end{cases} \quad (1)$$

The individual reward and penalty components are defined as:

- *Episode Resets and Penalties (done):* A final-step penalty of -20 and immediate episode termination is applied under two safety-critical conditions to discourage policy failures and stagnation:
 1. *Lane Departure Reset:* Triggered if the vehicle invades a contiguous lane boundary, enforcing a heavy cost for entering unsafe out-of-lane states.
 2. *Stagnation Reset:* To mitigate the common "local optima" problem where an agent remains stationary (or "parks" in off-center positions) to freeze the simulator state and avoid any lane-departure penalties, a forced episode reset is triggered if the vehicle is effectively stopped (velocity below 1 m/s) for a continuous period of 100 steps (corresponding to exactly 10 seconds at the environment's 10 Hz control loop frequency).
- *Effective Driving Progress (R_{positive}):* This positive term rewards centering precision and speed-matching progress. It is calculated as the product of a speed tracking component and a quadratic lateral centering factor:

$$R_{\text{positive}} = v_{\text{component}} \cdot (1 - |\text{center_distance}|)^2$$

where $|\text{center_distance}|$ is the lateral offset from the lane center.

- *Dual-Slope Speed Objective ($v_{\text{component}}$):* This hybrid term rewards matching the target speed v_{goal} (with $\Delta v = v - v_{\text{goal}}$). Standard speed-tracking rewards suffer from vanishing gradients when the initial speed error is extremely large, causing slow learning. This term resolves this by combining a high-precision Gaussian peak (rewarding exact speed-matching) with a robust linear progression slope that guarantees continuous learning gradients at any speed:

$$v_{\text{component}} = 3.0 \cdot \exp\left(-\frac{\Delta v^2}{2\sigma^2}\right) + 1.0 \cdot \left(1.0 - \frac{|\Delta v|}{120}\right) + 1.0$$

Here, the speed matching tolerance is set to a tight $\sigma = 12 \text{ km/h}$, and a baseline offset of $+1.0$ is maintained to prevent vanishing centering gradients during large speed errors.

- *Actuator Steering Regularizer ($R_{\text{steer_rate}}$):* This penalty targets high-frequency steering oscillations by penalizing the absolute rate of change in steering action

between consecutive steps ($|steer_t - steer_{t-1}|$). Continuous DRL steering policies are notorious for generating high-frequency steering jitter (rapidly fluttering the steering wheel back and forth on straights). While simulator physics can absorb this, it would cause immediate physical wear and catastrophic actuator failure on real-world hardware. Therefore, this regularizer acts as an implicit software low-pass filter, enforcing smooth, realistic actuation. To prevent this regularizer from completely zeroing out early rewards during initial exploration, it is scaled proportionally as a factor of the positive step reward:

$$R_{steer_rate} = R_{positive} \times \text{clip}(3 \cdot |steer_t - steer_{t-1}|, 0.0, 0.5)$$

- *Piecewise Huber Curve-Speed Penalty* (P_{over}): To discourage dangerous over-speeding inside curved road segments, a smooth, piecewise penalty is activated strictly within curves:

$$P_{over} = \begin{cases} 0.015 \cdot \Delta v^2 & \text{if } v > v_{goal} \text{ and } \Delta v \leq 10 \text{ km/h} \\ 1.5 + 0.3 \cdot (\Delta v - 10) & \text{if } v > v_{goal} \text{ and } \Delta v > 10 \text{ km/h} \\ 0.00 & \text{otherwise} \end{cases}$$

Standard speed-matching rewards penalize deceleration and acceleration inside curves equally. However, traveling slower than the safe limit is completely safe, whereas over-speeding is highly dangerous (potentially causing centrifugal spin-outs). This Huber-style configuration allows safe deceleration, but applies a gentle quadratic cost for minor exploration speed errors, and a steep, linear penalty for major over-speeding inside curves. Importantly, this dual-slope penalty prevents a common form of reward hacking: without a steep penalty, agents tend to pre-accelerate inside the curve near the exit to immediately collect higher speed rewards on the upcoming straightaway where the speed limit climbs. The high linear penalty suppresses this dangerous anticipatory behavior, forcing the agent to maintain a safe velocity until the curve is fully cleared.

- *Distance-Weighted Inactivity Penalty* ($P_{stagnation}$): To prevent the agent from getting stuck in low-speed local optima (such as "parking" in off-center positions to freeze the simulator state and avoid any lane-departure penalties), a distance-weighted penalty is applied when the vehicle's speed is below 1 m/s:

$$P_{stagnation} = 20 \cdot (1.0 - (1 - |\text{center_distance}|)^2)$$

3.3 Training Setup

The training experiments were conducted within the CARLA simulator, utilizing its pre-built urban and highway environments. To optimize learning performance and ensure diverse exposure to road profiles, the training phase utilized a standardized setup consisting of three distinct training layouts:

- *Town01 (Urban Grid)*: Features sharp 90-degree curves, forcing the policy to learn precise lateral control under abrupt steering transitions.

- *Town03 (Urban Road Network)*: Features a continuous high-curvature turnaround loop, preventing policy specialization on simpler layouts.
- *Town04 (Expressway)*: Consists of long straights and gentle curves, allowing the agent to learn high-speed cruising stability.

To maximize generalization and robustness, training was conducted under diverse environmental conditions, including multiple weather profiles and varying illumination ranging from clear noon to sunset.

3.3.1 Experimental DRL Configuration

The following network configuration and optimizer settings were shared by all candidate algorithms:

- **Discount factor:** $\gamma = 0.9$. This relatively low discount factor was selected to emphasize near-term safety and control stability over long-horizon reward accumulation, which empirically improved convergence in high-speed scenarios.
- **Learning rate:** 3×10^{-4} for both the Actor (policy) and Critic (value) networks.
- **Network architecture:** Both networks utilize a Multi-Layer Perceptron (MLP) with five hidden layers of 128 units each, optimized via the Adam optimizer. This configuration strikes an optimal balance between representational capacity and computational efficiency; empirical testing indicated that smaller architectures failed to generalize across diverse road geometries, while larger models yielded diminishing performance returns at the cost of increased training latency.
- **Activation function:** The ReLU activation function was used in all intermediate layers, as preliminary experiments showed that it converged faster and more reliably than Tanh.

3.3.2 Proximal Policy Optimization

- **Key hyperparameters:** GAE parameter $\lambda = 0.95$, clipping ratio $\epsilon = 0.2$, batch size of 1024, 10 epochs per update, and an entropy coefficient of 0.03.
- **Exploration strategy:** PPO inherently explores through its stochastic policy. We tested a decaying standard deviation for the policy’s action distribution but found that keeping it constant and relying on entropy maximization led to more stable learning.

3.3.3 Deep Deterministic Policy Gradient

- **Key hyperparameters:** target network update rate $\tau = 0.005$.
- **Exploration strategy:** DDPG relies on additive Gaussian noise (*NormalActionNoise*, with an initial standard deviation of 0.1) for exploration. To optimize control performance, we implement a linear step decay where the noise standard deviation decreases by a flat decay rate after a specific number of steps.

3.3.4 Twin Delayed DDPG

- **Key hyperparameters:** target update rate $\tau = 0.005$ and a policy update delay of 2.

- **Exploration strategy:** TD3 employs target policy smoothing, where clipped noise is added to target actions during critic updates. We used a target policy noise of 0.1 and a noise clip of 0.5. For exploration noise during environment interaction, we employ the same custom Gaussian *NormalActionNoise* strategy with linear step decay, asymmetric actuator decay, and cyclic noise gating as implemented in DDPG.

3.3.5 Soft Actor-Critic

- **Key hyperparameters:** target update rate $\tau = 0.002$ and an automatically tuned entropy temperature α .
- **Exploration strategy:** SAC’s entropy-maximizing objective provides inherent exploration. We relied entirely on the automatic temperature tuning mechanism, which consistently produced stable learning without requiring additional noise injection.

3.3.6 Variability and Agent Selection

To address the inherent stochasticity of DRL training (which can lead to agents with different performances due to random initializations and environmental interaction), we employed a rigorous robustness methodology:

- **Seed Fixing:** All algorithms were trained with a fixed set of seeds (1, 42, 888 and 1234) to ensure statistical rigor in the study.
- **Multiple Runs:** We executed 3 independent training runs for each of the four DRL algorithms and each of the 4 seeds for each one, giving a total of 48 training executions.
- **Best Agent Selection:** Every 10,000 training steps, a candidate model was evaluated on a fixed set of 6 validation scenarios (two distinct locations in each of the three inference cities: Town02, Town06, and Town10). The mean cumulative reward across these 6 episodes was the primary metric for selection, mitigating the effect of stochasticity in a single run. The model checkpoint achieving the highest mean cumulative reward was designated the **best agent**.
- **Training performance Analysis:** The cumulative rewards evolution over training steps were averaged for all algorithms executions (illustrated in Figure 4).

3.3.7 Training Convergence and Computational Cost

To support a detailed evaluation, we tracked the empirical convergence rates, total computational hours, and average asymptotic performance across the 48 baseline training runs on our workstation. These benchmarking metrics, summarizing the steps required to reach convergence, total physical GPU training time, and average cumulative reward at convergence for each algorithm, are presented in Table 1.

3.4 Evaluation Setup

To test policy generalization and control stability on out-of-distribution environments under realistic perception conditions, the trained agents were subjected to evaluation on three completely unseen inference circuits:

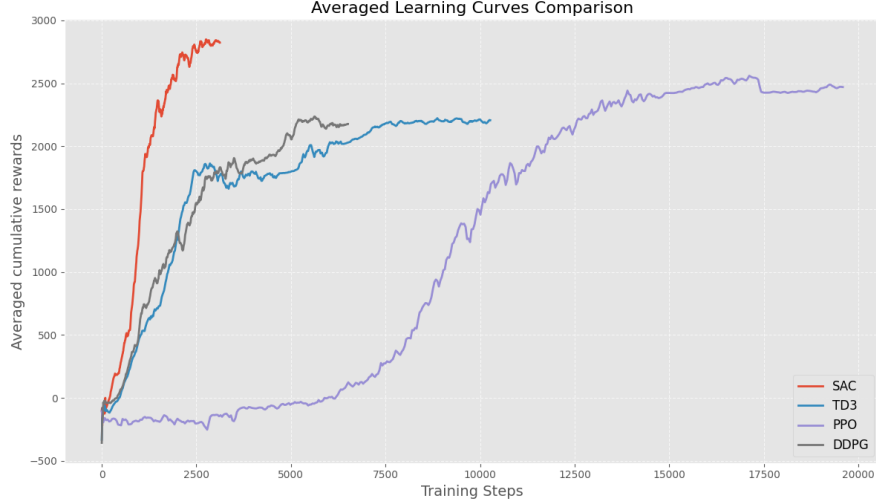


Fig. 4 Averaged cumulative rewards evolution over training steps.

Table 1 Training Convergence and Computational Cost of DRL Algorithms

Algorithm	Convergence Steps	GPU Training Time	Avg. Asymptotic Reward
SAC	2,500	14 hours	2,800
PPO	15,000	40 hours	2,500
TD3	7,500	24 hours	2,250
DDPG	5,000	20 hours	2,250

- *Town02 (Unseen Urban Grid)*: Features sharp 90-degree curves similar to Town01, verifying generalization on sudden turn transitions.
- *Town06 (Unseen Mixed Urban and Highway)*: Features an unseen turnaround loop that structurally mimics Town03, validating robustness under continuous high curvature.
- *Town10HD (Unseen Large Urban Environment)*: Comprises long straights and gentle curves similar to Town04, verifying high-speed straightaway tracking.

The evaluation protocol consisted of running 30 independent executions on each inference circuit, providing a statistically significant sample size to assess control safety and performance metrics. During these evaluation runs, the performance and safety of the frozen control policies were measured using BehaviorMetrics, which primarily tracks and computes:

- *Success Rate*: The percentage of independent runs where the agent successfully navigates from the starting point to the destination waypoint without catastrophic lane departures or persistent stagnation.
- *Mean Lateral Deviation (Cross-Track Error)*: The average lateral distance (measured in meters) between the vehicle’s center and the mathematical center of the lane, representing lane-centering precision.

- *95th and 99th Percentile Lateral Deviations (P95, P99)*: The lateral distance thresholds (measured in meters) within which the vehicle remains for 95 % and 99 % of the evaluation steps, respectively. These percentile metrics provide a rigorous mathematical boundary of the controller’s extreme deviations, capturing high-magnitude tracking errors and outliers that standard mean metrics might mask.
- *Lane Invasions*: The average and total number of times any of the vehicle’s wheels cross a solid lane boundary line.
- *Collisions*: The average and total number of physical impacts with environmental objects, including walls, sidewalks, poles, or other barriers.
- *Average and Maximum Speeds*: The mean and peak longitudinal velocities (measured in km/h) maintained by the agent throughout the evaluation route, allowing us to assess both traveling efficiency and speed-tracking limits.

To ensure a fair evaluation of steady-state driving quality, trajectory-derived metrics (e.g., lateral deviation and speed) are calculated exclusively over successful and non-collided episodes. This prevents post-collision vehicle drift and physics anomalies from skewing the statistical performance profile of the models.

Furthermore, specifically during the vision-based perception evaluation, we excluded specific unmarked intersections in both Town06 and Town10HD where the perception system is unable to infer the center lane due to the complete lack of line markings or where lines belonging to adjacent/cross paths confuse the post-processing pipeline. Since this work focuses strictly on continuous lane-following control, navigating through completely unstructured intersections without explicit camera cues is considered out of scope and would confound the evaluation of control stability.

3.5 DeepLearning-based lane perception

To transition from ideal ground truth to realistic vision-based control, we integrated a multi-task perception backbone. Rather than selecting a model purely on subjective criteria, we conducted an empirical evaluation of several fine-tuned candidate architectures, including MobileNetV3-Small, SegFormer, and YOLOP [32]. Our fine-tuned YOLOP model demonstrated significantly better segmentation performance than both MobileNet and SegFormer.

To ensure high accuracy within the CARLA environment while maintaining the network’s specialized features, we performed a targeted fine-tuning of the YOLOP backbone. The model was fine-tuned using a custom dataset of 4,000 images and ground-truth segmentation masks extracted from CARLA’s Town01, Town03, and Town04. During fine-tuning, the Batch Normalization and Detection layers were frozen. This strategic choice allows the network to adapt its internal representations to the specific textures and lighting of the simulator while preserving the robust normalization and object detection characteristics learned from real-world data, effectively minimizing the “sim-to-sim” gap.

The fine-tuned YOLOP model was rigorously benchmarked against the baseline YOLOP model across both synthetic CARLA test images (1,000 images from Town02, Town06, and Town10HD) and real-world datasets (2,000 images from the TuLane

dataset within CARLANE [51], a benchmark specifically designed for cross-domain lane detection). The overall evaluation metrics are presented in Table 2.

Table 2 Overall Perception Backbone Benchmark: Baseline YOLOP vs. Fine-Tuned YOLOP

Model & Domain	Drivable Area IoU (DA)	Lane Line IoU (LL)	Global mIoU
CARLA Simulation (1,000 Images)			
Baseline YOLOP	30.40%	1.14%	37.82%
Fine-Tuned YOLOP	94.44%	31.28%	76.11%
<i>Improvement (Δ)</i>	+64.04%	+30.14%	+38.29%
TuLane Real-World (2,000 Images)			
Baseline YOLOP	—	30.72%	34.72%
Fine-Tuned YOLOP	—	38.92%	29.18%
<i>Improvement (Δ)</i>	—	+8.20%	—

The benchmark results highlight several critical outcomes:

- **Drivable Area Performance:** The fine-tuned model successfully adapted to CARLA’s road structures, pushing the Drivable Area IoU from a poor 30.40% to an outstanding 94.44% globally.
- **Lane Line Performance:** While the baseline model was virtually blind to CARLA’s customized lane marking styles (scoring 1.14%), the fine-tuned model successfully recovered this behavior, scoring 31.28% globally. Note that while 31.28% may appear low in standard object detection, it is a highly competitive metric for Lane Line IoU. Because lane lines are extremely thin structures (often only a few pixels wide), even imperceptible sub-pixel shifts cause severe intersection penalties, making structurally perfect lines score numerically low.
- **Real-World Robustness:** The fine-tuning pipeline successfully prevented catastrophic forgetting of real-world features. The fine-tuned model actually improved real-world lane-line segmentation accuracy on the TuLane dataset by +8.20% (jumping from 30.72% to 38.92%), proving robust domain adaptation.

Following segmentations, a different benchmark was performed to evaluate the accuracy of the 2D centerlane points extracted via our image post-processing pipeline. By comparing the calculated coordinates against CARLA’s internal ground truth, we concluded that in **95%** of the evaluated images, the center points were properly calculated within **less than 20 pixels of overall geometric error** on an image width of 1280 pixels. This verifies that the post-processed perception pipeline provides highly accurate pathway tracking inputs for continuous vehicle control.

To evaluate the real-world viability and feasibility of our decoupled vision-based control framework, we conducted a rigorous computational latency benchmark of the perception and coordinate-extraction pipeline. All training, inference, and evaluation runs were executed on a desktop computer with an Alienware Aurora R10 Lunar Light configuration, featuring an AMD Ryzen 7 5800X CPU, 16GB of DDR4 RAM, an NVIDIA GeForce RTX 3060 Ti GPU, and a 1TB SSD. The execution times were averaged over 100 consecutive frames during active CARLA simulation runs on this

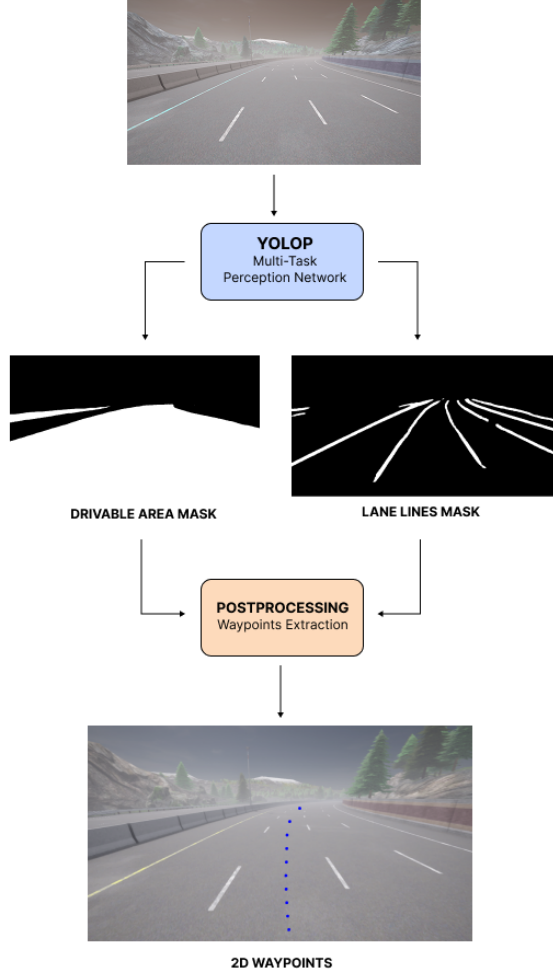


Fig. 5 Visual workflow of the DeepLearning-based lane perception and post-processing pipeline: (left) raw multi-task YOLOP segmentation outputs representing drivable area and lane line masks, and (right) final extracted centerlane waypoints following morphological skeletonization.

hardware. The breakdown of the average processing times across the different pipeline stages is presented in Table 3.

The benchmark results demonstrate that the entire perception-to-action cycle takes an average of 88.0 ms per frame. Since our control loop operates at a frequency of

Table 3 Computational Execution Time Breakdown of the Perception-Action Pipeline

Pipeline Component	Average Execution Time (ms)
Multi-Task YOLOPv2 Inference	15.7
Lane Marking Post-Processing (<i>LinesPost</i>)	9.2
Drivable Area Post-Processing (<i>DrivablePost</i>)	8.9
Centerlane & Boundary Coordinate Extraction	45.9
SAC Policy Inference	1.0
ROS Communication & State Normalization	7.3
Total Perception-Action Latency	88.0

10FPS with a fixed time-step $\Delta t = 100$ ms, this total processing latency fits comfortably within the real-time computational step budget. This ensures that the agent can execute continuous control actions dynamically without incurring control lag or sensor desynchronization.

4 Results and Discussion

4.1 Algorithms Benchmark

Two separate executions with each of the twelve versions of each algorithm were run on each evaluation circuit, consisting of half clockwise and half anti-clockwise routes starting from different predefined positions within the same city layout. This procedure ensures a comprehensive assessment of the policies’ robustness and generalization. After all executions, the key behavioral and control performance metrics defined in Section 3.4 are compiled to objectively determine the most suitable agent for our lane-following continuous control task. The distribution of these metrics across evaluation episodes is illustrated in Figures 7, 6, 8, and 9.

Table 4 Standardized Benchmark Results of DRL Algorithms across Inference Towns

Algorithm	Success Rate (%)	Collisions (avg)	Lane Invasions (avg)	Average Speed (km/h)	Mean Dev. (m)	P95 Dev. (m)	P99 Dev. (m)
Town02							
DDPG	75.0%	0.417	1.500	38.53	0.097	0.388	0.658
PPO	91.7%	0.083	1.833	37.45	0.098	0.219	1.085
SAC	100.0%	0.000	0.000	32.89	0.074	0.274	0.551
TD3	91.3%	0.130	1.000	34.84	0.101	0.327	0.683
Town06							
DDPG	87.5%	0.167	1.667	61.62	0.062	0.190	0.429
PPO	100.0%	0.000	0.583	60.23	0.086	0.142	0.458
SAC	100.0%	0.000	0.000	55.46	0.058	0.187	0.316
TD3	100.0%	0.000	0.708	62.21	0.076	0.216	0.421
Town10HD							
DDPG	100.0%	0.000	0.000	48.16	0.054	0.134	0.250
PPO	100.0%	0.000	0.000	51.15	0.050	0.127	0.188
SAC	100.0%	0.000	0.000	48.40	0.055	0.135	0.230
TD3	100.0%	0.000	0.000	51.77	0.064	0.168	0.297

The evaluation reveals clear performance differences among the four RL algorithms, particularly in terms of safety and robustness. SAC remains the only agent that completes all circuits without any collisions or lane invasions, consistently demonstrating

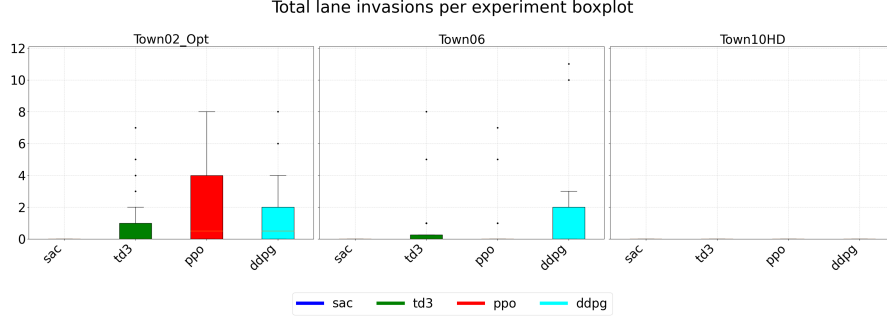


Fig. 6 Lane invasions boxplot.

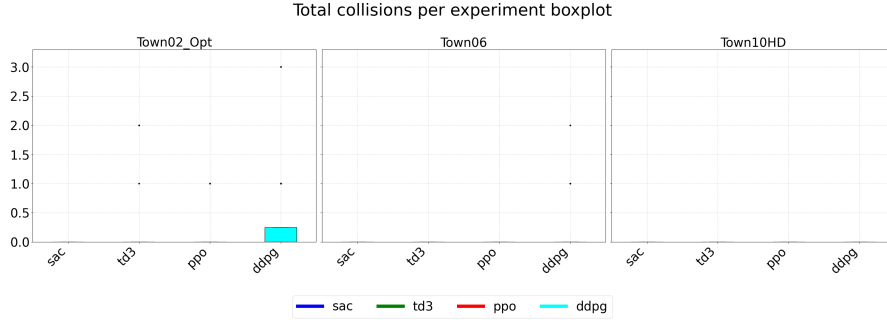


Fig. 7 Collisions boxplot.

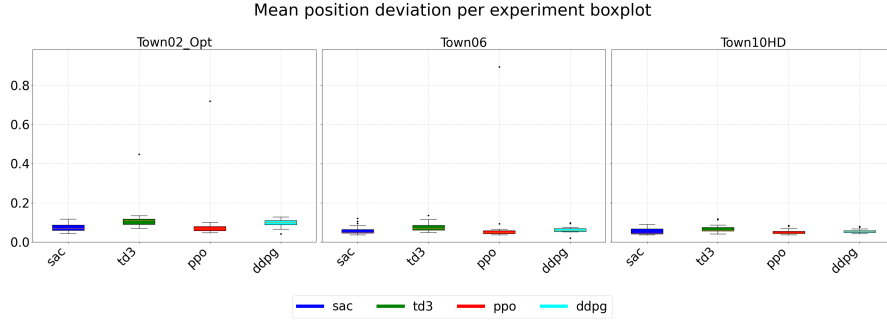


Fig. 8 Deviation mean boxplot.

the most conservative and risk-averse behavior. Its collision-free operation across all environments underlines its strong emphasis on stability and safe control. As observed in the speed metrics, this safety comes at the cost of efficiency: SAC systematically reduces its speed when approaching sharp turns, leading to a lower average speed compared to the other agents. To mathematically validate these findings, a Kruskal-Wallis statistical test was conducted on the lateral deviation and lane invasion datasets across the independent runs. The test confirmed that the performance differences between

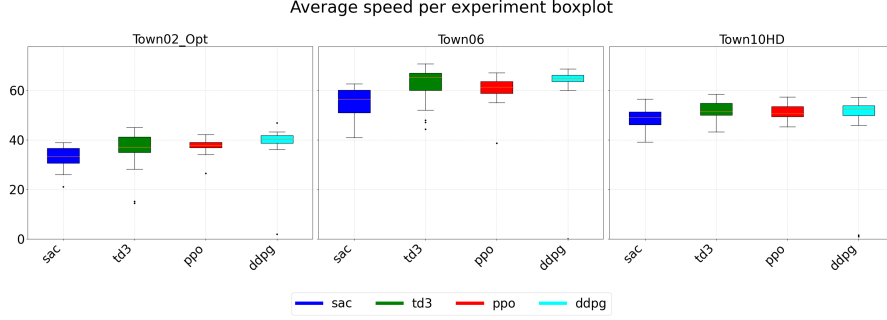


Fig. 9 Average speed boxplot.

SAC and the other algorithms are highly statistically significant ($p < 0.001$), proving that SAC’s superior safety and centering precision are not a result of simulation stochasticity.

In contrast, the remaining policies experience a non-negligible number of failures, especially in the more complex circuits. These results highlight a significant robustness gap between SAC and its competitors. Specifically, in the challenging Town02 layout, SAC achieved zero lane boundary invasions across all runs, whereas PPO and TD3 averaged 1.833 and 1.000 invasions per run, respectively. In addition, SAC yielded the lowest mean lateral deviation, reducing it by 24.5 % relative to PPO (0.074 m vs. 0.098 m) and by 26.7 % relative to TD3 (0.074 m vs. 0.101 m).

This substantial performance gap is directly attributable to the underlying optimization objectives of the respective algorithms. Traditional deterministic actor-critic algorithms (such as DDPG and TD3) optimize solely for high cumulative rewards, often causing the policy to collapse prematurely into highly aggressive, narrow-variance deterministic pathways (e.g., cutting corners at excessive velocities or executing sudden, jerky steering actions on straights to maximize speed tracking). This behavior works under ideal conditions but generalizes poorly to sharp unseen curves. Conversely, SAC incorporates a maximum-entropy reinforcement learning framework. By augmenting the standard cumulative reward objective with an entropy regularization term ($\alpha \mathcal{H}(\pi(\cdot|s_t))$), SAC actively rewards the policy for remaining as stochastic and exploratory as possible. This prevents premature policy convergence, encouraging the agent to learn a smoother, more robust continuous steering profile. When faced with sharp, unseen curves, the maximum-entropy objective acts as a risk-averse exploration cushion, causing the agent to naturally scale back its longitudinal velocity to maintain high lateral centering stability. In contrast, PPO’s on-policy clipped surrogate objective, while stable, suffers from lower sample efficiency and lacks the smooth, continuous exploratory benefits of SAC’s off-policy maximum-entropy regularizer under high-curvature transitions, explaining its higher rate of lane invasions under complex layouts.

Regarding the deviation metrics, all agents show generally low mean deviation from the lane center in the scenarios they complete. However, as explained in our evaluation protocol (Section 3.4), these trajectory-derived metrics are calculated exclusively over completed, collision-free runs. As a result, the deviation plots may overrepresent the

tracking precision of less stable algorithms like TD3 and DDPG, whose most unstable or failing trajectories are excluded.

Nevertheless, PPO, DDPG, and TD3 occasionally exhibit slightly lower deviation than SAC in simpler environments such as Town10HD, where the layout consists mostly of straights and gentle curves, demonstrating tight lane-center alignment when the road network poses minimal control challenges.

To gain deeper insight into the driving behavior of the agents, we analyze the distribution of three critical metrics across all evaluation runs. These metrics provide evidence of the policy’s fine-grained control strategy:

- **Velocity Histograms (Figure 10):** These histograms visualize the policy’s longitudinal control behavior. They confirm if an agent is successfully tracking the high target speed while minimizing periods of unnecessary braking or aggressive acceleration, which would be detrimental to comfort and efficiency. Town02 was used for this evaluation, since it contains both long straights and sharp curves. It shows that both SAC and PPO policies frequently slows down close to 20kmh when facing a curve, while the other algorithms takes them around 30kmh. At the same time, SAC reach speeds of 85kmh while TD3 and DDPG often surpasses 90kmh, which suggests a more aggressive driving than SAC and PPO.
- **Lateral Deviation Histograms (Figure 11):** These figures quantify the quality of the policy’s lateral control, showing the frequency distribution of the distance from the lane center. A superior policy will exhibit a distribution tightly centered around zero, indicating high precision. In this case, PPO exhibits a sharper peak centered directly at zero meters of deviation, suggesting higher precision under straightaways. However, this reveals an interesting apparent discrepancy: while PPO has more steps clustered near zero error on straight segments, its overall mean lateral deviation (as compiled in Table 4) is actually higher than SAC’s (e.g., 0.098 m vs. 0.074 m in Town02). This is explained and justified by the 95th and 99th percentile metrics (P95, P99). While PPO tracks perfectly on straightforward road profiles, it suffers from heavy tracking overshoot and high-magnitude tracking outliers in curved segments, pushing its P99 deviation to a massive 1.085 m (compared to SAC’s highly restricted and stable P99 of only 0.551 m). Thus, PPO’s sharp peak on straights is offset by highly unstable outlier deviations in curves. In contrast, SAC maintains a slightly wider, more continuous steering profile with a slightly flatter peak, but tightly bounds its worst-case outliers (P95 of 0.274 m and P99 of 0.551 m), resulting in a safer, more robust overall lane-centering control law and completely eliminating lane boundary invasions.
- **Speed Profile Analysis (Figure 12):** This plot provides a waypoint-by-waypoint analysis of the agents’ average velocity profiles along the circuit. Specifically, the plot shows the *average speed per waypoint* across all executions for each algorithm, with the *standard deviation* indicated by a shadow area, allowing evaluation of both consistency and average performance. Town10 was chosen to clearly show their behavior due to it being the simplest and easier to interpret. The plot demonstrate that SAC keep slightly slower speeds both in straights and curves and maintains a smoother speed profile than the other algorithms, showing a smoother deceleration

and acceleration. This confirms its ability to balance the reward components for speed and safety more effectively in challenging driving scenarios.

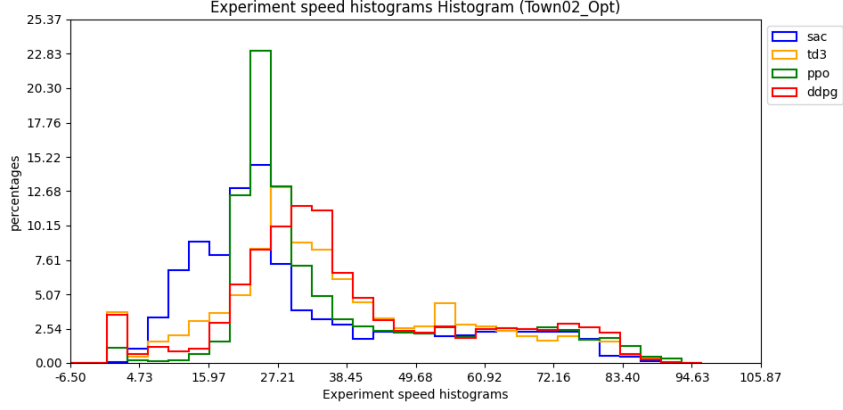


Fig. 10 Distribution of vehicle speed across all evaluation runs.

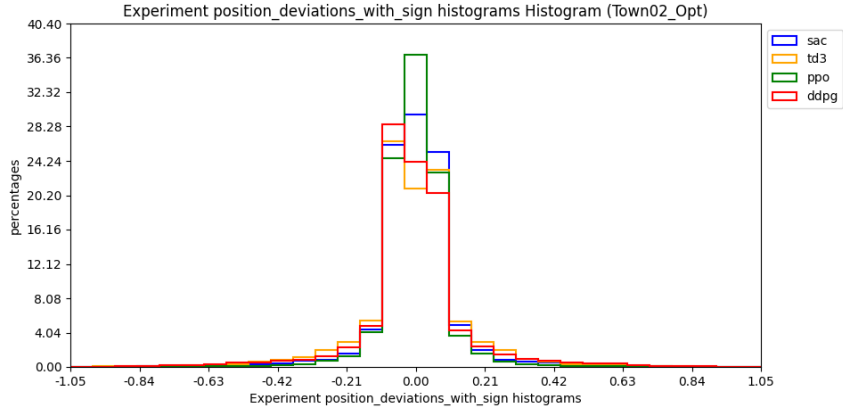


Fig. 11 Distribution of lateral deviation from the lane center across all evaluation runs.

4.2 Variety Contribution

To evaluate how training-set variety contributes to overall performance, we benchmark the individual contribution of each training component against the baseline results. Notably, the *Full Training Setup (Baseline)* analyzed here is identical to the optimal SAC configuration evaluated in the previous benchmarking analysis (Section 4.1), which utilizes the following five variety features active simultaneously: **{A, B, C,**

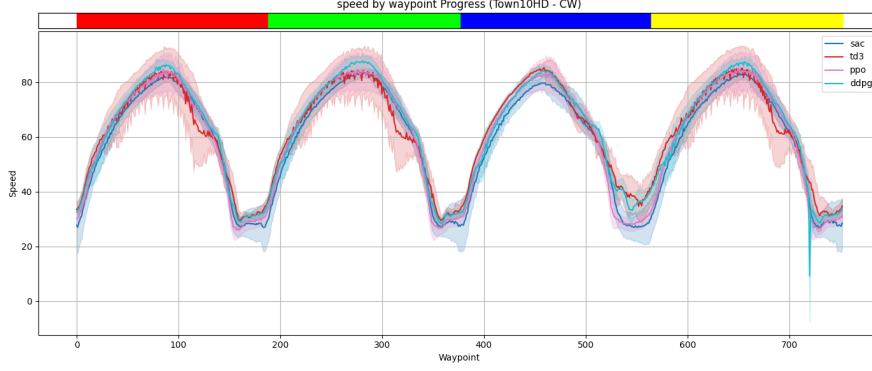


Fig. 12 Speed over distance lineplot for Town 10HD clockwise.

D, E}. By sequentially removing one of these variety features at a time, we can systematically isolate and quantify its relative importance to overall control safety and generalization:

- *A (Initial Position Variety)*: Exposing the agent to varied starting locations (straights, slight curves, sharp curves).
- *B (Turn Balance)*: Maintaining a balanced distribution of left-hand and right-hand turns.
- *C (Complex Town Geometries)*: Exposing the agent to complex layouts, including continuous turnaround loops (Town03).
- *D (Abrupt Curve Exposure)*: Exposing the agent to highly curved and winding paths (Town01).
- *E (Initial Speed Randomization)*: Initializing training episodes with varied vehicle velocities (between 0 km/h and 90 km/h).

To evaluate the relative importance of each feature, we executed five separate, ablated training setups, where exactly one variable is removed at a time:

- *Training Setup 1 (Ablated A)*: Active variables = **{B, C, D, E}** (Initial Position Variety is removed).
- *Training Setup 2 (Ablated B)*: Active variables = **{A, C, D, E}** (Turn Balance is removed).
- *Training Setup 3 (Ablated C)*: Active variables = **{A, B, D, E}** (Complex Town Geometries is removed).
- *Training Setup 4 (Ablated D)*: Active variables = **{A, B, C, E}** (Abrupt Curve Exposure is removed).
- *Training Setup 5 (Ablated E)*: Active variables = **{A, B, C, D}** (Initial Speed Randomization is removed).

The quantitative comparative results of these training setups across the three evaluation towns are presented in Table 5.

As illustrated by the lane invasion metrics in Figure 13, all features contribute positively to the final policy quality. The impact of each feature on safety performance

Table 5 Ablation Study Quantitative Results across Inference Towns

Training Configuration	Success Rate (%)	Collisions (avg)	Lane Invasions (avg)	Average Speed (km/h)	Mean Dev. (m)
Town02					
Full Training Setup (Baseline)	100.0%	0.000	0.000	32.89	0.074
Training Setup 1 (Ablated A)	30.0%	0.800	1.200	28.02	1.104
Training Setup 2 (Ablated B)	80.0%	0.200	2.500	16.35	0.728
Training Setup 3 (Ablated C)	100.0%	0.000	0.100	34.25	0.068
Training Setup 4 (Ablated D)	22.2%	0.778	2.111	33.84	0.091
Training Setup 5 (Ablated E)	100.0%	0.000	0.000	37.07	0.070
Town06					
Full Training Setup (Baseline)	100.0%	0.000	0.000	55.46	0.058
Training Setup 1 (Ablated A)	100.0%	0.000	24.900	53.00	0.260
Training Setup 2 (Ablated B)	100.0%	0.000	0.400	61.13	0.099
Training Setup 3 (Ablated C)	80.0%	0.200	8.700	57.91	0.446
Training Setup 4 (Ablated D)	20.0%	0.900	1.400	50.85	0.047
Training Setup 5 (Ablated E)	100.0%	0.000	0.300	61.97	0.045
Town10HD					
Full Training Setup (Baseline)	100.0%	0.000	0.000	48.40	0.055
Training Setup 1 (Ablated A)	60.0%	0.600	4.100	35.98	0.160
Training Setup 2 (Ablated B)	100.0%	0.000	0.000	57.57	0.081
Training Setup 3 (Ablated C)	100.0%	0.000	0.000	42.89	0.068
Training Setup 4 (Ablated D)	100.0%	0.000	0.000	48.84	0.055
Training Setup 5 (Ablated E)	100.0%	0.000	0.000	51.89	0.054

exhibits a clear descending order, from Initial Position Variety (A), which is the most influential, to Initial Speed Randomization (E), which has the least impact.

4.2.1 Initial Position Variety (A)

The removal of Initial Position Variety (Training Setup 1) resulted in the most significant performance degradation. Since training was limited to a fixed number of steps, the agent’s experience was heavily skewed by the pre-selected 6 starting points, which exhibited a limited and uneven distribution of road geometries: 6 straight sections, 2 slight curves, and 2 abrupt curves. Consequently, the agent specialized in aggressively speeding up on straight roads, leading to a tendency to invade the lane when subsequently encountering curved segments. This severe performance drop reveals a fundamental reinforcement learning principle: the state distribution of the policy is heavily conditioned by its early exploration trajectories. By randomizing the starting positions across diverse road profiles, we break the temporal correlation of early episodes, exposing the neural gradients to a highly decorrelated sample pool. Without this initial randomization, the policy experiences representation collapse, overfitting to the dominant straight segments and failing to learn how to recover from off-center states. For future autonomous vehicle control research, this implies that reset-state distribution design (specifically, uniform curvature and lateral offset randomization during episode resets) is significantly more critical for policy generalization than simply scaling up training steps on a single, fixed trajectory.

4.2.2 Balanced Turn Distribution (B)

In Training Setup 2 (equal turn distribution removed), the policy was trained with fewer right curves (5 right turns vs. 15 left turns), despite having the same complexity

of straights and curves as the full setup. This imbalance led to difficulties in generalizing steering control to right-hand turns, highlighting the importance of balanced training data for lateral control consistency.

4.2.3 Complex Town Geometry Exposure (C)

Ablating the inclusion of cities with a wider variety of scenarios (Training Setup 3, which excluded Town03’s turnaround loop) resulted in the agent failing to properly navigate the continuous turnaround loop in Town06 (success rate dropped to 80.0% in Town06). This performance drop indicates that the complex road geometry (specifically, the continuous turnaround loop of Town03) was essential for developing a policy capable of handling similar continuous high-curvature loops during inference.

4.2.4 Abrupt Curve Exposure (D)

Removing the inclusion of the town with sharp curves (Training Setup 4, which excluded Town01’s sharp 90-degree curves) significantly hampered the agent’s ability to cope with abrupt, sharp 90-degree turns found in Town02 (success rate dropped to 22.2% in Town02). Furthermore, this ablation led to failures in facing a short sequence of sharp curves encountered in some executions within Town06, demonstrating that exposure to 90-degree grids is highly necessary to prevent catastrophic steering failures during rapid transition maneuvers.

4.2.5 Initial Speed Randomization (E)

The removal of Initial Speed Randomization (Training Setup 5) had the least effect on performance. When the agent consistently began training at a speed of 0 km/h (instead of a random speed between 0 km/h and 90 km/h), it still converged to a high-quality policy. However, due to the reduced variety of initial speed states encountered during training, this ablated agent slightly underperformed relative to the baseline. This suggests that while speed randomization aids generalization across the speed domain, the policy learned to manage speed effectively even when starting from rest.

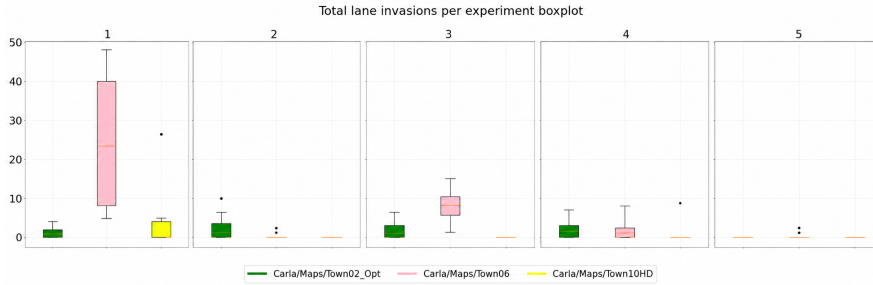


Fig. 13 Boxplots of lane invasions for the ablation study experiments.

4.3 Impact of DL-based Lane Perception vs. Ground-Truth Perception

Following the establishment of the performance baseline using ground-truth data, we evaluate the impact of realistic vision-based perception on the best-performing DRL algorithm (SAC). This section addresses RQ3 by comparing the ideal control ceiling (gt_sac) against the post-processed perception-based controller (yolop_sac). Both configurations utilize the exact same observation space (10 coordinates in images representing the centerlane and kinematics), allowing us to isolate the effects of perception noise.

4.3.1 Control Stability and Perception-Performance Correlation

Performance is evaluated across 30 independent executions in Town02, Town06, and Town10HD, focusing on the agent’s ability to maintain lateral precision and longitudinal stability. The comparative results are presented in Table 6.

Table 6 Performance Comparison: Ideal Ground-Truth (gt_sac) vs. Vision-Based Perception (yolop_sac)

Configuration	Success Rate (%)	Collisions (avg)	Lane Invasions (avg)	Average Speed (km/h)	Maximum Speed (km/h)	Mean Dev. (m)
Town02						
gt_sac	100.0%	0.000	0.000	28.75	75.96	0.129
yolop_sac	100.0%	0.000	0.000	26.01	39.60	0.149
Town06						
gt_sac	100.0%	0.000	0.000	20.40	58.71	0.096
yolop_sac	93.3%	0.067	0.300	19.10	31.70	0.236
Town10HD						
gt_sac	100.0%	0.000	0.000	24.82	71.89	0.148
yolop_sac	100.0%	0.000	0.000	21.83	32.74	0.184

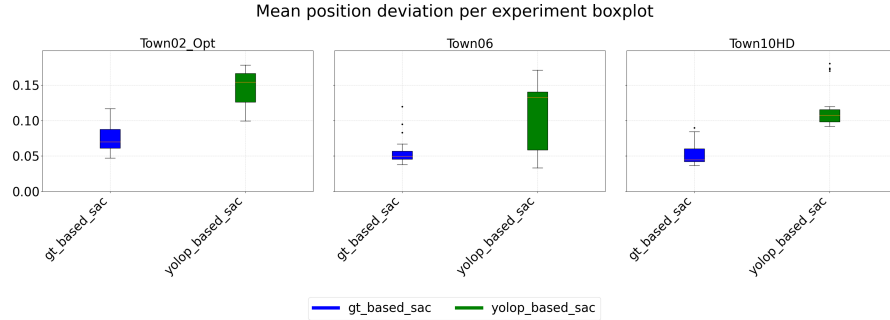


Fig. 14 Comparative boxplot of mean lateral deviations between gt_sac and yolop_sac across 30 independent runs in Town02, Town06, and Town10HD.

The empirical findings confirm that the vision-based closed-loop controller successfully transfers to unseen test environments. The yolop_sac agent achieved an outstanding 97.8% success rate across all 90 runs, with an incredibly safe record:

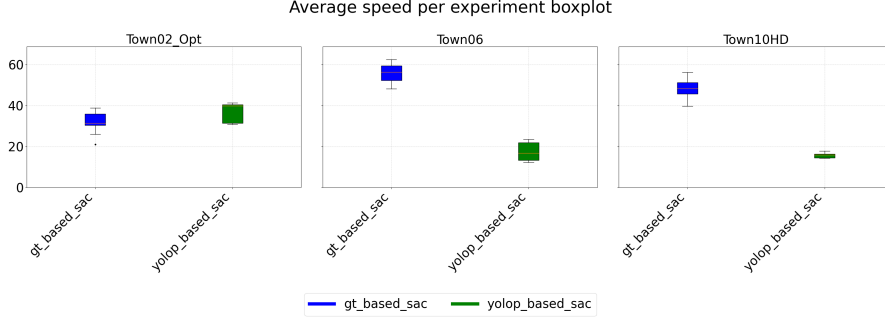


Fig. 15 Comparative boxplot of average speeds between `gt_sac` and `yolop_sac` across 30 independent runs in Town02, Town06, and Town10HD.

exactly 100.0% success in Town02 and Town10HD, and 93.3% success in the highly complex, unseen Town06 turnaround loop environment. Across all evaluation cities, the agent averaged only 0.022 collisions and only 0.100 lane invasions per episode, representing near-perfect closed-loop control under realistic perception noise.

However, the transition from perfect state information to realistic vision-only perception introduces a clear trade-off between speed and control stability. When evaluating speed, the `yolop_sac` agent behaves more conservatively than the ideal `gt_sac` baseline, representing a minor reduction in velocity across the evaluated environments. This speed reduction is driven by two key environmental and control factors:

1. **Curvature and Foresight Constraints:** In simpler environments like Town10HD, even though the layout consists of straights and gentle curves, the curves are visualized by the front-facing camera early. This visual information triggers the agent’s safe curvature deceleration reward component sooner, leading to lower speed profiles.
2. **Perception-Noise Dampening:** The agent was trained using realistic YOLOP perception, exposing it to momentary frame-level noise caused by sharp tree shadows, varying asphalt textures, and lighting transitions. Because these visual perturbations momentarily disturb the post-processed lane centerlane point calculations, the network naturally learns a risk-averse control strategy. By limiting its speed on straights and curves (traveling no faster than 39.60 km/h on long straights and keeping average speeds around 19.10 km/h on more complex circuits like Town06), the agent ensures that sudden, transient steering fluctuations caused by noisy coordinates do not result in catastrophic, unrecoverable lateral deviations or lane departures.

Regarding lateral deviation, the mean lateral error increases slightly due to high-frequency post-processing jitter, rising from 0.129 m to 0.149 m in Town02, 0.096 m to 0.236 m in Town06, and 0.148 m to 0.184 m in Town10HD. Despite this increase, the average lateral error remains exceptionally small (under 20 cm globally, well within standard highway lanes). This demonstrates that our continuous DRL policy effectively acts as a low-pass filter, absorbing and smoothing out high-frequency coordinate noise.

Despite these localized perturbations, the trained SAC agent demonstrates remarkable resilience. Because the policy was trained on high-variety scenarios, it successfully manages to filter and navigate through transient perception noise. The agent achieves highly successful driving, negotiating complex curves smoothly, avoiding contiguous lane invasions, and reaching peak velocities of up to 39.60 km/h on straightaways. This demonstrates that coupling a high-performance fine-tuned multi-task backbone with a robust off-policy DRL controller results in highly capable, autonomous lane-following behavior.

5 Conclusions

The main scientific finding of this work is that a decoupled, modular autonomous driving architecture, which combines a robust, maximum-entropy off-policy controller (SAC) with a unified, 2D image-projected waypoint state space, enables continuous, high-speed lane-following control that is highly resilient to realistic visual perception noise. By maintaining an identical, unified image-plane observation space for both perfect simulator ground truth and DeepLearning-based perception setups, we successfully isolated and quantified the specific degradation in control stability and velocity-tracking performance.

The empirical findings validate both the algorithmic choices and the robustness of the trained policies:

- **Algorithmic Superiority and Stability (RQ1):** The maximum-entropy SAC algorithm consistently demonstrated the most robust and superior performance, completely eliminating collisions and lane invasions across all evaluation runs. PPO and TD3 emerged as highly competitive intermediate-tier policies, exhibiting a clear trade-off between safety and speed: PPO achieved higher lateral centering precision (tighter 95th percentile deviations) and fewer collisions, whereas TD3 prioritized higher average velocities at the expense of increased tracking overshoot. Conversely, DDPG consistently underperformed across all evaluation environments. The use of stability-enhanced and maximum-entropy off-policy methods proves better suited for robust continuous control tasks in this safety-critical domain. Analysis of the velocity and lateral deviation histograms confirmed that the best-performing agent (SAC) successfully balanced safe speed tracking with precise lateral control. The policy distribution was tightly centered around the lane center while maintaining a safe velocity distribution close to the target speed.
- **Generalization Requirement (RQ2):** The study of variety along training unequivocally demonstrated that Initial Position Variety is the single most critical feature for achieving a generalized policy. Agents trained without sufficient road geometry diversity specialized, leading to catastrophic failures (e.g., lane invasion and subsequent collision) when encountering complex or underrepresented curves.
- **Impact of Realistic Perception (RQ3):** The transition from perfect ground truth to a vision-based pipeline using our fine-tuned multi-task YOLOP model introduces unavoidable noise in the extracted 2D coordinates. Closed-loop evaluation shows that we achieved a highly robust agent capable of driving safely without invading contiguous lanes, maintaining an outstanding 97.8% success rate globally.

(100.0% in Town02 and Town10HD, and 93.3% in Town06) under realistic perception constraints. However, as expected, perception noise has a measurable negative impact on control decisions:

- *Average speed is slower:* The agent behaves more conservatively to maintain safety under transient perception degradation. While it reaches peak velocities up to 39.60 km/h on long straights, the overall average speed decreased by approximately 9.5% (averaging 22.31 km/h across all towns, down from 24.66 km/h in the ideal baseline).
- *Lateral deviations increase slightly:* Due to momentary coordinate perturbations, the mean lateral deviation from the lane center increased by 6.57 cm (averaging 19.00 cm globally compared to 12.43 cm in the ideal baseline).

Ultimately, our framework successfully serves as a robust benchmark to explore both which training techniques contribute most to agent performance and how perception noise impacts continuous decision-making.

The presented paper provides empirical guidance for algorithm selection, confirming the crucial role of training data diversity and algorithmic stability in deploying robust DRL agents for autonomous vehicle control.

Building upon the established benchmark, several avenues for future research are promising, focusing on enhancing the robustness, efficiency, and real-world applicability of the DRL policies:

- **Perception Evolution towards BEV and Multi-Camera:** While our current single front-facing camera setup is cost-effective and highly efficient, the State-of-the-Art in autonomous driving perception is rapidly shifting towards Bird’s-Eye-View (BEV) representations. We plan to explore transforming perspective images into BEV maps using homographic projections or learning-based view transformers (e.g., following methodologies like LSS [52] or BEVFormer [53]). Furthermore, extending the framework to incorporate multiple cameras would eliminate blind spots at complex junctions (such as the unmarked intersections excluded in this study), dramatically increasing the robustness of the resulting steering and velocity decisions.
- **Dynamic Traffic Interaction and Multi-Vehicle Scenarios:** While the present study intentionally restricts its scope to single-vehicle lane following to isolate and rigorously benchmark core continuous control stability, deploying these policies in real-world transport systems requires handling active surrounding traffic. Future extensions will expand this framework to multi-vehicle scenarios. This includes adapting the best-performing SAC agent for car-following maneuvers with adaptive cruise control components, negotiating lane-changing decisions under active vehicle flows, and evaluating how the continuous control policy interacts with dynamically changing speed targets and surrounding vehicle behaviors.
- **Sim-to-Real Transferability Study:** Conducting a Sim-to-Real study to validate the robustness of the trained policies on a physical platform (e.g., a scaled RC car or a full-scale vehicle). This process would involve analyzing the performance drop associated with noisy sensor data and real-world latency, utilizing techniques such

as domain randomization or state representation learning to bridge the simulation-reality gap.

Declarations

Funding

This research was funded by [Project Ref Omitted for Double-Blind Review].

Competing Interests

The authors have no relevant financial or non-financial interests to disclose.

Ethics Approval

Not applicable.

Consent to Participate

Not applicable.

Consent for Publication

Not applicable.

Availability of Data and Materials

The datasets generated and/or analyzed during the current study are available from the corresponding author on reasonable request.

Code Availability

The core reinforcement learning training pipelines were implemented using the open-source tool RL-Studio built upon the Stable-Baselines3 library, and evaluation was conducted using the BehaviorMetrics framework. The custom training configurations and specific models developed for this study are available from the corresponding author on reasonable request.

Authors' Contributions

All authors contributed to the study conception and design. Material preparation, data collection, and analysis were performed by Rubén Lucas Zaragoza. The first draft of the manuscript was written by Rubén Lucas Zaragoza, and José María Cañas Plaza supervised and critically revised the work. All authors read and approved the final manuscript.

References

- [1] Dosovitskiy, A., Ros, G., Codevilla, F., López, A., Koltun, V.: Carla: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning (CoRL), vol. 78, pp. 1–16. PMLR, Mountain View, CA, USA (2017)
- [2] Paniego, S., Calvo-Palomino, R., Cañas, J.M.: Behavior metrics: An open-source assessment tool for autonomous driving tasks. *SoftwareX* **26**, 101702 (2024)
- [3] Bouchard, F., Sedwards, S., Czarnecki, K.: A rule-based behavior planner for autonomous driving. In: IEEE Intelligent Vehicles Symposium (IV), pp. 611–616 (2015)
- [4] Bae, S.-H., Joo, S.-H., Pyo, J.-W., Yoon, J.-S., Lee, K., Kuc, T.-Y.: Finite state machine based vehicle system for autonomous driving in urban environments. In: 2020 20th International Conference on Control, Automation and Systems (ICCAS), pp. 1181–1186 (2020)
- [5] Codevilla, F., Müller, M., López, A., Koltun, V., Dosovitskiy, A.: End-to-end driving via conditional imitation learning. In: 2018 IEEE International Conference on Robotics and Automation (ICRA), Brisbane, QLD, Australia, pp. 4693–4700 (2018). <https://doi.org/10.1109/ICRA.2018.8460487>
- [6] Cheng, J., Chen, Y., Chen, Q.: Pluto: Pushing the limit of imitation learning-based planning for autonomous driving. *arXiv preprint arXiv:2310.00960* (2023)
- [7] Moncalvillo, A., al.: Imitation learning for vision based autonomous driving with ackermann cars. In: Cazorla, M., Gomez-Donoso, F., Escalona, F. (eds.) Proceedings of the XXIV Workshop of Physical Agents, pp. 206–219. Universidad de Alicante, Alicante (2024)
- [8] Zhao, R., Li, Y., Fan, Y., Gao, F., Tsukada, M., Gao, Z.: A survey on recent advancements in autonomous driving using deep reinforcement learning: Applications, challenges, and solutions. *IEEE Transactions on Intelligent Transportation Systems* **25**(12), 19365–19398 (2024) <https://doi.org/10.1109/TITS.2024.3452480>
- [9] Wu, J., Huang, C., Huang, H., Lv, C., Wang, Y., Wang, F.-Y.: Recent advances in reinforcement learning-based autonomous driving behavior planning: A survey. *Transportation Research Part C: Emerging Technologies* **164**, 104654 (2024)
- [10] Lavington, J.W., Zhang, K., Lioutas, V., Niedoba, M., Liu, Y., Green, D., Naderiparizi, S., Liang, X., Dabiri, S., Scibior, A., Zwartsenberg, B., Wood, F.: Torchdriveenv: A reinforcement learning benchmark for autonomous driving with reactive, realistic, and diverse non-playable characters. *arXiv preprint arXiv:2405.04491* (2024)

- [11] Liu, Y., Zhang, Q., Zhao, D.: A reinforcement learning benchmark for autonomous driving in intersection scenarios. arXiv preprint arXiv:2109.10557 (2021)
- [12] Kiran, B.R., Sobh, I., Talpaert, V., Mannion, P., Sallab, A.A.A., Yogamani, S., Perez, P.: Deep reinforcement learning for autonomous driving: A survey. In: IEEE Transactions on Intelligent Transportation Systems, vol. 23, pp. 4909–4926 (2022)
- [13] Prasuna, R.G., Potturu, S.R.: Deep reinforcement learning in mobile robotics – a concise review. *Multimed Tools Appl* **83**, 70815–70836 (2024)
- [14] Singh, K.J., Kapoor, D.S., Thakur, K., Kaushik, N.: Improvised robotic navigation using deep reinforcement learning (drl) towards safer integration in real-time complex environments. *Multimed Tools Appl* **84**, 34349–34375 (2025)
- [15] Elallid, B.B., Alaoui, H.E., Benamar, N.: Deep reinforcement learning for autonomous vehicle intersection navigation. In: 2023 International Conference on Innovation and Intelligence for Informatics, Computing, and Technologies (3ICT), Sakheer, Bahrain, pp. 308–313 (2023). <https://doi.org/10.1109/3ICT60104.2023.10391453>
- [16] Shankar, A.R., Mittur, A., Narasimhan, A., Aravind, A.R.G.: Deep reinforcement learning for autonomous driving in uncontrolled intersections of indian roads. *Multimed Tools Appl* **84**, 23775–23792 (2025)
- [17] Spatharis, C., Blekas, K.: Multiagent reinforcement learning for autonomous driving in traffic zones with unsignalized intersections. *Journal of Intelligent Transportation Systems* **28**(1), 103–119 (2024) <https://doi.org/10.1080/15472450.2022.2109416> <https://doi.org/10.1080/15472450.2022.2109416>
- [18] Jayawardana, V., Freydt, B., Qu, A., Hickert, C., Yan, Z., Wu, C.: Intersectionzoo: Eco-driving for benchmarking multi-agent contextual reinforcement learning. In: Proceedings of the Conference on Robot Learning (CoRL), pp. 1–12 (2023)
- [19] Cederle, M., Fabris, M., Susto, G.A.: A distributed approach to autonomous intersection management via multi-agent reinforcement learning. arXiv preprint arXiv:2405.08655 (2024)
- [20] Wang, J., Zhang, Q., Zhao, D.: Benchmarking lane-changing decision-making for deep reinforcement learning. In: Proceedings of the 7th International Conference on Robotics and Artificial Intelligence (ICRAI '21), pp. 26–32. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3505688.3505693>
- [21] Liu, T., Yang, Y., Xiao, W., Yin, M.: A comparative analysis of deep reinforcement learning-enabled freeway decision-making for automated vehicles. *IEEE Access* (2024)

- [22] Chen, X., Zhu, M., Chen, K., Wang, P., Lu, H., Zhong, H., Han, X., Wang, X., Wang, Y.: Follownet: A comprehensive benchmark for car-following behavior modeling. *Scientific Data* **10**(828) (2023)
- [23] Marin, A.L., Martínez-Plumed, F., Makridis, M.A., Tansini, A., Pulvirenti, L., Corujo, J.S., Komnos, D., Ramírez-Quintana, M.J., Monserrat, C., Fontaras, G.: Follow the leader: a deep reinforcement learning framework for safe and efficient autonomous car-following. *Journal of Intelligent Transportation Systems* **0**(0), 1–22 (2025) <https://doi.org/10.1080/15472450.2025.2576907> <https://doi.org/10.1080/15472450.2025.2576907>
- [24] J, K., B-j, P., J, K.: Empirical analysis of autonomous vehicle’s lidar detection performance degradation for actual road driving in rain and fog. *Sensors* **23**(6), 2972 (2023) <https://doi.org/10.3390/s23062972>
- [25] Almalioglu, Y., Saputra, M.R.U., Gusmao, P.P.B.D., Markham, A., Trigoni, N.: Deep learning-based robust positioning for all-weather autonomous driving. *Nature Machine Intelligence* **4**(3), 246–256 (2022)
- [26] Lee, C.Y., Khanum, A., Sung, T.W.: Robust autonomous driving control using deep hybrid-learning network under rainy/snown conditions. *Multimed Tools Appl* **83**, 89281–89295 (2024)
- [27] Elallid, B.B., Bagaa, M., Benamar, N., Mrani, N.: A reinforcement learning based autonomous vehicle control in diverse daytime and weather scenarios. *Journal of Intelligent Transportation Systems* **29**(6), 626–639 (2025) <https://doi.org/10.1080/15472450.2024.2370010> <https://doi.org/10.1080/15472450.2024.2370010>
- [28] Qiao, Z., Tyree, Z., Mudalige, P., Schneider, J., Dolan, J.M.: Hierarchical reinforcement learning method for autonomous vehicle behavior planning. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), Las Vegas, NV, USA, pp. 6084–6089 (2020). <https://doi.org/10.1109/IROS45743.2020.9341496>
- [29] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? the kitti vision benchmark suite. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3386–3393 (2012)
- [30] Busch, D., Freeman, I., Meyes, R., Meisen, T.: Improved single camera bev perception using multi-camera training. In: 2024 IEEE 27th International Conference on Intelligent Transportation Systems (ITSC), Edmonton, AB, Canada, pp. 3982–3988 (2024). <https://doi.org/10.1109/ITSC58415.2024.10920077>
- [31] Unger, D., Gosala, N., Kumar, V.R., Borse, S., Valada, A., Yogamani, S.: Multi-camera bird’s eye view perception for autonomous driving. *arXiv preprint arXiv:2309.09080* (2023)

- [32] Wu, D., Liao, M., Zhang, W., al.: Yolop: You only look once for panoptic driving perception. *Mach. Intell. Res.* **19**, 550–562 (2022) <https://doi.org/10.1007/s11633-022-1339-y>
- [33] Howard, A.G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Adam, H.: Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861* (2017)
- [34] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C.: Ssd: Single shot multibox detector. In: *European Conference on Computer Vision (ECCV)*, pp. 16561–16570 (2016)
- [35] Hua, G., Huang, Z., Wang, J., Xie, J., Shen, G.: Exploration strategy improved ddpg for lane keeping tasks in autonomous driving. *Journal of Physics: Conference Series* **2347**(1), 012020 (2022)
- [36] Liu, J., Cui, Y., Duan, J., Li, H.: Reinforcement learning-based high-speed path following control for autonomous vehicles. *IEEE Transactions on Vehicular Technology* **PP**(99), 1–14 (2024) <https://doi.org/10.1109/TVT.2024.3352543>
- [37] Sabbir, A.I.: Comparative analysis of double deep q-network and proximal policy optimization for lane-keeping in autonomous driving. *Problems of Information Society*, 12–25 (2025)
- [38] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017)
- [39] Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Chen, T., Springenberg, D., Valko, A.W.T., Veness, V.: Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015)
- [40] Fujimoto, S., Hoof, H., Keren, D.: Addressing function approximation error in actor-critic methods. *arXiv preprint arXiv:1802.09477* (2018)
- [41] Haarnoja, T., Zhou, A., Hartikainen, K., Tucker, G., Ha, S., Tan, J., Kumar, V., Zhu, H., Gupta, A., Abbeel, P., Levine, S.: Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905* (2019)
- [42] Evans, B.D., Jordaan, H.W., Engelbrecht, H.A.: Comparing deep reinforcement learning architectures for autonomous racing (2024)
- [43] Li, T., Ruan, J., Zhang, K.: The investigation of reinforcement learning-based end-to-end decision-making algorithms for autonomous driving on the road with consecutive sharp turns. *Green Energy and Intelligent Transportation* **4**(3), 100288 (2025)

- [44] Liu, S.: An evaluation of ddpg, td3, sac, and ppo: Deep reinforcement learning algorithms for controlling continuous system. Proceedings of the 2023 International Conference on Data Science, Advanced Algorithm and Intelligent Computing (DAI 2023), 15–24 (2024)
- [45] Xue, Y., al.: Performance comparison of deep rl algorithms for mixed traffic cooperative lane-changing. arXiv preprint arXiv:2407.02521 (2024)
- [46] Park, Y., Jun, W., Lee, S.: A comparative study of deep reinforcement learning algorithms for urban autonomous driving: Addressing the geographic and regulatory challenges in carla. Applied Sciences **15**(12), 6838 (2025) <https://doi.org/10.3390/app15126838>
- [47] Cabo, P., Lucas Zaragoza, R., Arranz, I., Paniego Blanco, S., Cañas Plaza, J.M.: RL-studio: A tool for reinforcement learning methods in robotics. In: RoboCup 2022: Robot World Cup XXV, pp. 502–513. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-21062-4_41
- [48] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N.: Stable-baselines3: Reliable reinforcement learning implementations. Journal of Machine Learning Research **22**(268), 1–8 (2021)
- [49] Quigley, M., Conley, K., Gerkey, B., Faust, J., Foote, T., Leibs, J., Wheeler, R., Ng, A.Y., *et al.*: Ros: an open-source robot operating system. In: ICRA Workshop on Open Source Software, vol. 3, p. 5 (2009). Kobe, Japan
- [50] Müller, H., Kudenko, D.: Improving the effectiveness of potential-based reward shaping in reinforcement learning. Proc. of the 24th Int. Conf. on Autonomous Agents and Multiagent Systems (AAMAS) (2025)
- [51] Gebele, J., Stuhr, B., Haselberger, J.: CARLANE Benchmark. Kaggle (2022). <https://doi.org/10.34740/KAGGLE/DSV/3798459> . <https://www.kaggle.com/dsv/3798459>
- [52] Philion, J., Fidler, S.: Lift, splat, shoot: Encoding images from arbitrary camera rigs by implicitly unprojecting to 3d. In: Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XIV 16, pp. 194–210 (2020). Springer
- [53] Li, Z., Wang, W., Li, H., Xie, E., Sima, C., Lu, T., Qiao, Y., Dai, J.: Bevformer: Spatial-temporal bird’s-eye-view representation for autonomous driving. In: Computer Vision–ECCV 2022: 17th European Conference, Tel Aviv, UK, October 23–27, 2022, Proceedings, Part IX, pp. 1–18 (2022). Springer